PROJEK ALGORITMA

SMART CAMPUS NAVIGATION AND SHORTEST PATH SYSTEM

Dosen pengampu :
Dr.Eng. Ir. Aji Ery Burhandenny ST., M.AIT.

Disusun Oleh:

Maizan Arifin	(25051030009)
Habibi Aulia Rizki	(25051030010)
Nicholas Candra Pascalis	(25051030011)
Rahmatullah Wahyu Nugroho	(25051030038)
Juan Ramadhani	(25051030039)

TEKNIK ELEKTRO
DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS NEGERI YOGYAKARTA

2026

DAFTAR ISI

DAFTAR ISI..... 2
BAB I PENDAHULUAN..... 3
A. Pembukaan..... 3
B. Latar Belakang..... 3
BAB II PERANCANGAN SISTEM..... 4
A. Arsitektur Sistem..... 4
B. Penjelasan Algoritma..... 4
Algoritma yang digunakan:..... 5
Struktur Data yang digunakan:..... 6
BAB III IMPLEMENTASI..... 6
A. Lingkungan Pengembangan..... 6
B. Struktur Data Berbasis Linked List..... 6
C. Graf dan Direktori Gedung..... 6
D. Algoritma Pencarian..... 6
E. Antar muka penggunaan berbasis CLI..... 7
BAB IV ANALISIS..... 8
A. Analisis Big-O..... 8
BAB V PENGUJIAN DAN PEMBAHASAN..... 9
A. Hasil Uji..... 9
BAB VI JAWABAN PERTANYAAN ANALISIS..... 10
A. Pertanyaan..... 10
B. Jawaban..... 10
BAB VII PENUTUP..... 16
A. Kesimpulan..... 16

BAB I
PENDAHULUAN

A. Pembukaan

Puji syukur atas kehadirat Tuhan Yang Maha Kuasa yang telah melimpahkan hidayah serta inayahnya sehingga kami dapat menyelesaikan project algoritma berdasarkan topik "Smart Campus Navigation and Shortest Path System" dengan baik dan tepat waktu.

Tugas ini disusun untuk memenuhi Tugas Project Algoritma pada mata kuliah Algoritma dan Struktur data. Dan kami mengucapkan terima kasih kepada dosen pengampu kami Bapak Dr. Eng. Ir. Aji Ery Burhandenny ST., M.AIT. yang telah memberikan bimbingan serta arahannya kepada kami

Dalam hal ini kami menyadari bahwa dalam penyusunan laporan ini masih jauh dari kata sempurna, baik dari segi efisiensi code algoritma maupun dokumen. Sehingga kami sangat berterima kasih apabila terdapat kritik serta saran yang bersifat membangun.

Semoga laporan Project ini mampu memberikan manfaat bagi pembaca sehingga dapat dijadikan referensi dalam perkembangan teknologi Smart Campus

B. Latar Belakang

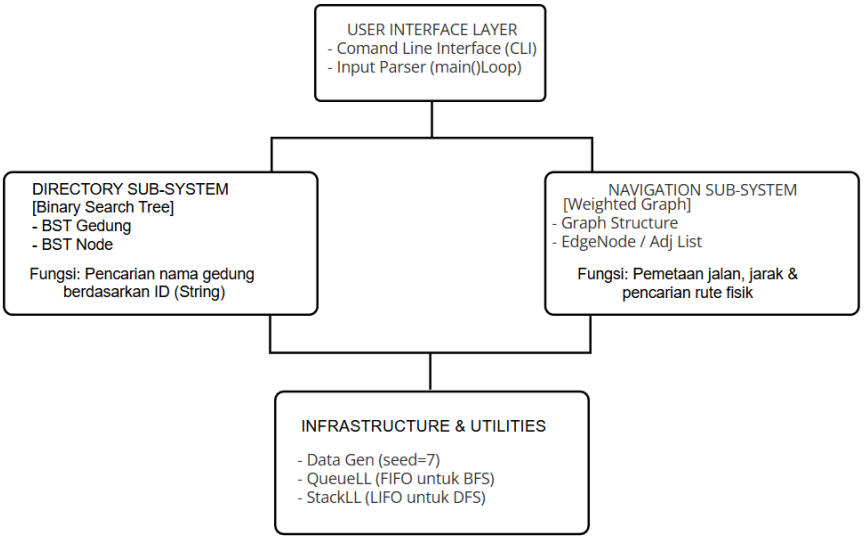
Dalam era digital saat ini, semakin banyak perkembangan yang mulai bermunculan dengan berbagai perkembangan infrastruktur terintegrasi, gedung perkuliahan, perpustakaan, masjid, dan bangunan lainnya yang dapat menunjang berlangsungnya proses belajar mengajar bagi lingkungan kampus. Bagi dosen, mahasiswa maupun pengunjung sering kali mendapatkan tantangan yaitu mengenai tata ruang kampus.

Dalam pencarian rute yang cepat dan efisien menggunakan peta konvensional sudah dianggap tidak relevan lagi. Oleh karenanya, dikembangkanlah konsep Smart Campus Navigation and Shortest Path System sebagai navigasi digital yang cerdas. Navigasi yang baik tidak cukup hanya menampilkan gambar saja tetapi harus memperhatikan kalkulasi matematis untuk memberikan jalur yang cepat serta efisien.

BAB II
PERANCANGAN SISTEM

A. Arsitektur Sistem

DIAGRAM_ARSITEKTUR
==== SMART CAMPUS NAVIGATION SYSTEM - ARCHITECTURE BLUEPRINT
====



==========

Rincian Aliran Data & Algoritma:

1. DIJKSTRA : Menghitung bobot kumulatif (meter) rute terpendek graf.
2. BFS & DFS : Menelusuri seluruh node menggunakan QueueLL dan StackLL kustom.
3. INORDER : Mengurutkan indeks data gedung alfabetis pada struktur BST.

==========

```
def main():
    print(DIAGRAM_ARSITEKTUR)

if __name__ == "__main__":
    main()
```

B. Penjelasan Algoritma

Algoritma adalah urutan langkah-langkah logis dan terstruktur yang digunakan untuk menyelesaikan suatu masalah. Dalam pemrograman, algoritma berfungsi sebagai dasar atau prosedur kerja program agar dapat menghasilkan output sesuai tujuan. Pada program Smart Campus Navigation System, algoritma

digunakan untuk mengelola data gedung, mencari jalur tercepat, melakukan penelusuran graph, dan mencari data gedung dengan efisien.

Algoritma yang digunakan:

1. Algoritma Dijkstra

   Algoritma Dijkstra adalah algoritma pencarian jalur terpendek pada graph berbobot. Algoritma ini bekerja dengan mencari jarak minimum dari satu node sumber ke node lainnya. Fungsinya untuk mencari jalur tercepat antar gedung pada sistem navigasi kampus.

   Cara Kerja
   - Menentukan node awal.
   - Memberikan nilai jarak awal 0 pada node sumber dan tak hingga pada node lain.
   - Memilih node dengan jarak terkecil.
   - Memperbarui jarak tetangga jika ditemukan jalur lebih pendek.
   - Proses diulang sampai semua node selesai diperiksa.
2. Algoritma BFS (Breadth First Search)

   Algoritma BFS adalah algoritma penelusuran graph yang bekerja secara melebar atau level. Fungsinya untuk menjelajahi graph, mengecek konektivitas gedung, mencari gedung terisolasi.

   Cara Kerja
   - Node awal dimasukkan ke queue.
   - Node dikunjungi satu per satu.
   - Semua tetangga node dikunjungi terlebih dahulu sebelum masuk ke level berikutnya.
3. Algoritma DFS (Depth First Search)

   Algoritma DFS algoritma penelusuran graph yang bekerja secara mendalam hingga node terakhir sebelum kembali ke node sebelumnya. Fungsinya untuk menjelajahi seluruh graph secara mendalam.

   Cara Kerja
   - Mengunjungi node awal.
   - Masuk terus ke node tetangga terdalam.
   - Jika tidak ada jalan lagi, kembali ke node sebelumnya.
4. Algoritma BST (Binary Search Tree)

   Algoritma BST struktur data pohon biner yang digunakan untuk pencarian data secara cepat berdasarkan urutan tertentu. Fungsinya untuk mencari data gedung berdasarkan kode gedung.

   Cara Kerja
   - Membandingkan data yang dicari dengan root.
   - Jika lebih kecil → ke kiri.
   - Jika lebih besar → ke kanan.
   - Proses dilakukan sampai data ditemukan.

Struktur Data yang digunakan:

1. Graph
   Menyimpan hubungan antar jalan (Edge) dan gedung (Node). Kegunaannya mempermudah proses navigasi dan pencarian jalur.
2. Queue
   Digunakan pada algoritma BFS. Cara Kerjanya data yang masuk pertama akan keluar pertama (FIFO).
3. Stack/Rekursi
   Digunakan pada DFS. Cara Kerjanya data terakhir yang masuk diproses terlebih dahulu (LIFO)

BAB III
IMPLEMENTASI

A. Lingkungan Pengembangan

Sistem Navigasi Kampus Cerdas diimpelemtasikan menggunakan bahasa pemograman python. Struktur data dan algoritma dari nol. Sistem ini dibangun tanpa bergantung kepada libraries eksternal. hal tersebut bertujuan untuk memastikan efisiensi memori, serta pemahaman mendalam terhadap struktur data.

B. Struktur Data Berbasis Linked List
   - QueueLL menggunakan 2 buah pointer yaitu head dan tail memastikan elemen queue dan dequeue berjalan konstan O(1). support BFS (Breadth-First Search).
   - StackLL menggunakan prinsip LIFO , dengam memasukkan data (push) dan mengeluarkan data (pop) sehingga menghasilkan kompleksitas O(1) support DFS (Depth-First search)

C. Graf dan Direktori Gedung, berbasis data nama gedung yang melalui 2 struktur data modular:
   - Graf Tak berarah berbobot(graph), menggunakan Adjacency List setiap gedung bertindak sebagai node di dalam sebuah dictionary. EdgeNode digunakan untuk merepresentasikan jalan
   - BSTGedung, untuk mengoptimalkan pencarian informasi gedung

D. Algoritma Pencarian
   - Algoritma Dijkstra, mencari rute terpendek dari gedung asal menuju seluruh gedung lainnya
   - Rekontrksi Jalur, bekerja setelah dijkstra memetakan jarak, muali dari gedung tujuan, mencari parent nya hingga kembali ke gedung asal
   - BFS dan DFS, BFS dengan QueueLL untuk graf melebar, sedangkan DFS dengan StackLL menelusuri sedalam mungkin.

- Terisolasi, Memanfaatkan BFS dengan titik awal A1, hasilnya berupa kumpulan seluruh gedung yang dapat dijangkau dari node A1. (Terisolasi= Semua gedung-gedung terjangkau)

E. Antar muka penggunaan berbasis CLI

Dalam fungsi main(). menggunakan struktur while True menunggu masukan teks dari pengguna. argumen commond code diubah menjadi besar melalui .upper(, sedangkan sisanya sebagai parameter fungsi. Dalam percabangan if elif else sebagai pengontrol lalu lintas instruksi. untuk memanggil fungsi-fungsi backed dijkstra(),BFS(),DFS() maupun fungsi manipulasi graf. sebagai jaminan program tidak crash saat pengguna salahj input format, seluruh blok CLI dibungkus dalam mekanisme penanganan (try-except).

BAB IV
ANALISIS

A. Analisis Big-O

- Struktur Data (Linked List,Queue,Stack)
  Class QueueLL dan StackLL
  Kompleksitas Waktu (enqueue) :O(1) tambah dan hanya mengubah pointer dan untuk (dequeue) O(1) untuk menghapus elemen
  Kompleksitas Ruang O(n) dimana n adalah jumlah elemen disimpan dalam queue atau stack
- Graph (Adjacency List)
  dalam code ini direpresentasikan dengan Linked List (EdgeNode)
  add_node(node_id,nama):O(1) rata-rata ditambahkan pada dictionary
  add_edge(u, v, bobot): O(1) input edge bari di head tidak perlu traversal sampai tail
  neighbors(u):O(deg(u)) waktu dan ruang, dimana deg(u) jumlah tetangga dari node u
- Algoritma Dijkstra
  Kompleksitas Waktu : O(V^2)), dimana V merupakan jumlah node dan E jumlah jalan(edge)
  dalam loop terkecil (min_dist) dengan literasi ke seluruh node di graph.adj proses minim O(V) sehingga total menjadi O(V^2)
- BFS dan DFS
  Kompleksitas waktu : O (V+E) setiap running akan masuk ke queue/stack dan akan dievaluasi ketika gedung didequeue.
  Komplesitas Ruang : O(V) menyimpan set visited
- BST(BSTGEDUNG)
  Insert dan Search
  average case : O (log V) waktu, dengan syarat data seimbang
  Worst Case : O(V) waktu, jika terdapat data yang sudah terurut pohon akan condong dan bekerja sesuai Linked List linier
  Inorder : O(V) waktu untuk mengunjungi setiap node
- Fungsi Utilitas dan CLI
  generate_edges : O(V) untuk spanning tree dan add V/2 edge acak
  reconstruct_path:O(V) waktu dan ruang digunakan untuk menelusuri pointer induk dari node tujuan kembali ke sumber, V merupakan panjang jalur
  Terisolasi : operasi set semua_gedung-terjangkau dengan O(V) rata-rata

BAB V
PENGUJIAN DAN PEMBAHASAN

A. Hasil Uji

- Graph dengan Adjacency List memetakan ID node ke Linked List (EdgeNode). Pendekatan ini sangat efisien secara memori untuk graf yang cenderung sparse (jarang) seperti peta kampus. Penambahan node dan edge beroperasi pada kompleksitas waktu O(1), di mana penambahan sisi baru langsung disisipkan pada head dari Linked List. Pada LAPORAN_GRAPH, tercatat 30 gedung dan 44 jalan

- Implementasi Queue untuk algoritma BFS dan Stack untuk algoritma DFS dibangun menggunakan Linked List khusus (NodeLL). Hal ini memastikan operasi push/enqueue dan pop/dequeue berjalan secara konstan O(1) tanpa risiko realokasi memori yang sering terjadi pada Dynamic Array.

- Untuk pencarian direktori gedung (CARI_GEDUNG), penggunaan BST menawarkan efisiensi waktu rata-rata O(log n) dibandingkan pencarian sekuensial (linear) O(n). Hasil pengujian menunjukkan sistem dapat langsung mengembalikan nilai "Gerbang Utama" saat key "A1" dimasukkan.

- Untuk pencarian jalur satu gedung ke gedung lain, Sistem menggunakan Algoritma Dijkstra murni dengan representasi array/dictionary sederhana untuk menyimpan jarak (dist). Kompleksitas waktu untuk implementasi ini adalah O(V^2 + E), di mana V adalah jumlah gedung dan E adalah jumlah jalan.

- Kegunaan adalah DFS Bergerak menelusuri graf sedalam mungkin melalui satu cabang sebelum mundur (backtracking). Hal ini cocok digunakan jika ingin mensimulasikan rute patroli berkeliling kampus yang panjang.

- BFS Menyebar ke seluruh tetangga terdekat terlebih dahulu (secara radial). Program ini sangat tepat menggunakan hasil dari BFS untuk fitur TERISOLASI. Dengan membandingkan kumpulan node yang terjangkau oleh BFS dari gerbang utama (A1) dengan total node yang ada, sistem dapat secara matematis menyimpulkan apakah ada graf yang terputus (terisolasi). Hasil pengujian menunjukkan set terisolasi kosong, membuktikan graf berhasil digenerasi sebagai Connected Graph

BAB VI
JAWABAN PERTANYAAN ANALISIS

A. Pertanyaan

1. Implementasi Dijkstra dengan array sederhana memiliki kompleksitas $O(V^2 + E)$. Untuk kampus dengan 100 gedung dan 300 koneksi, hitung total operasi perbandingan. Bandingkan dengan implementasi menggunakan min-heap [O((V+E) log V)]. Pada nilai V dan E berapakah perbedaannya menjadi signifikan (>10x)?
2. Adjacency list menggunakan Linked List menghemat memori dibanding adjacency matrix untuk sparse graph. Hitung memori adjacency matrix $O(V^2)$ vs adjacency list O(V+E) untuk 30 node, 55 edge. Jika kampus diperluas ke 300 node dan 400 edge (masih sparse), berapa penghematan memori dalam byte (asumsikan tiap elemen 8 byte)?
3. BFS menemukan jalur dengan jumlah hop minimum (tanpa bobot), sedangkan Dijkstra menemukan jalur dengan total bobot minimum. Berikan contoh konkret pada peta kampus Anda di mana kedua algoritma menghasilkan jalur BERBEDA. Jelaskan implikasi praktisnya bagi mahasiswa.
4. Jika ada node yang terisolasi (tidak terhubung ke komponen utama), Dijkstra akan mengembalikan dist[v] Bagaimana sistem navigasi harus menangani kasus ini secara user-friendly? Usulkan modifikasi pada fungsi dijkstra dan CLI tanpa mengubah kompleksitas Big-O dasar.
5. BST Direktori Gedung menggunakan kode gedung string sebagai kunci. Jika kode dimasukkan secara alfabetis (A1, A2, A3, ... Z9), BST menjadi linear. Jelaskan dampaknya dan usulkan satu pendekatan sederhana (tanpa AVL/Red-Black) untuk memitigasi masalah ini pada skenario kampus.

B. Jawaban

1. Diketahui:
   1) Jumlah Gedung V = 100
   2) Koneksi E = 300

   Penyelesaian:

- Dijkstra Array Sederhana

$$O(V^2 + E)$$
$$O(100^2 + 300)$$
$$\approx 10300 \text{ operasi}$$

- Dijkstra Min-Heap

$$O((V + E) \log V)$$

$$O((100 + 300) \log_2 (100)$$

$$\approx 2656 \text{ operasi}$$

- Perbandingan

$$\frac{10300}{2656} = 3.88$$

- Kapan Perbedaannya menjadi Signifikan (10x)?

$$\frac{V^2+E}{(V+E) \log V} > 10$$

Asumsi bahwa $E \approx V^2$, maka

$$\frac{V^2+V^2}{(V+V^2) \log V} > 10$$

$$\frac{2V^2}{V \log V} > 10$$

Note:

Karena tujuan analisis ini adalah melihat laju pertumbuhan dominan pada $V + V^2$ suku dominan memang V^2, tetapi implementasi heap yang praktisnya bertumbuh mendekati $V \log V$.

$$\frac{V^2}{V \log V} > 10$$

$$\frac{V+V}{V \log V} > 10$$

$$\frac{V}{\log V} > 10$$

- Mencoba Perbandingan Nilai > 10

$$V = 50$$

$$\frac{50}{\log_2 50} \approx \frac{50}{5.64} \approx 8.8$$

$$V = 70$$

$$\frac{70}{\log_2 70} \approx \frac{70}{6.13} \approx 11.4$$

Jadi perbedaan performa mulai signifikan (> 10x) sekitar $V = 70$

2. Perbandingan Efisiensi Memori: Adjacency Matrix vs Adjacency List

Peta kampus termasuk sparse graph (graf jarang) karena jumlah jalannya jauh lebih sedikit dibanding kapasitas maksimal ruang yang tersedia. Oleh karena itu, penggunaan Adjacency List jauh lebih hemat memori daripada Adjacency Matrix karena hanya mencatat jalan yang benar-benar ada.

a. Perhitungan pada Kampus Saat Ini (30 Node, 55 Edge)

- Adjacency Matrix $O(V^2)$
  - Total elemen: 30 * 30 = 900 elemen
  - Kebutuhan memori: 900 * 8 byte = 7.200 byte
- Adjacency List O(V + 2E):
  - Total elemen: 30 + 2(55) = 140 elemen.
  - Kebutuhan memori: 140 * 8 byte = 1.120 byte

b. Perhitungan Jika Kampus Diperluas (300 Node, 400 Edge)

- Adjacency Matrix $O(V^2)$:
  - Total elemen: 300 * 300 = 90.000 elemen
  - Kebutuhan memori: 90.000* 8 byte = 720.000 byte
- Adjacency List O(V + E):
  - Total elemen: 300 + 2(400) = 1.100 elemen.
  - Kebutuhan memori: 1.100 * 8 byte = 8.800 byte

Jika kampus diperluas menjadi 300 gedung (node) dan 400 jalan (edge), selisih memori antara kedua metode adalah:

720.000 byte - 8.800 byte= 711.200 byte

Sistem navigasi kampus ini menggunakan Undirected Graph (Graf Tidak Berarah) sehingga data jalan disimpan bolak-balik (dua kali). Meskipun demikian, penggunaan Adjacency List terbukti sangat efisien untuk sparse graph karena berhasil menghemat RAM sebesar 711.200 byte (sekitar 711 KB) dengan cara hanya mencatat rute jalan yang benar-benar ada tanpa membuang memori pada relasi kosong.

3.

```
A1        ----------        100        ----------        B2
 |                                                        |
```

10				10
|				|
C3	----------	10	----------	D4

Jika mencari jalur dari A1 ke B2

- BFS akan memilih $A1 \rightarrow B2$, karena hanya membutuhkan 1 hop, maka total jarak 100 meter.
- Dijkstra akan memilih $A1 \rightarrow C3 \rightarrow D4 \rightarrow B2$, karena total bobotnya lebih kecil, maka total jarak $10 + 10 + 10 = 30$. Walaupun jumlah hop lebih banyak.

Implikasi Praktis untuk Mahasiswa
Perbedaan ini berpengaruh pada navigasi di lingkungan kampus.

- Jika menggunakan BFS, mahasiswa mungkin mendapatkan jalur dengan perpindahan gedung paling sedikit, tetapi belum tentu jaraknya paling dekat.
- Jika menggunakan Dijkstra, mahasiswa mendapatkan jalur dengan jarak tempuh paling pendek sehingga lebih efisien dalam waktu dan tenaga.

Dalam sistem navigasi kampus, Dijkstra lebih cocok digunakan karena mempertimbangkan jarak sebenarnya antar gedung, sedangkan BFS lebih cocok digunakan jika semua jalan dianggap memiliki bobot yang sama.

4. Penanganan Metode Terisolasi secara User-Friendy pada sistem Navigasi

a. Sisi Pengguna (User-Friendly): Saat sistem mendeteksi gedung tujuan terisolasi (jarak tetap ∞), aplikasi wajib menyembunyikan teks error teknis seperti inf dan menggantinya dengan pesan informatif yang solutif bagi manusia, contoh: "Maaf, Gedung A saat ini tidak dapat diakses..."

b. Modifikasi Fungsi Djikstra, Deteksi rute terputus dilakukan tanpa merusak algoritma dasar $O(V^2+E)$, penanganan ini cukup diterapkan berupa pengecekan kondisi sederhana pada fungsi reconstruct path atau saat hasil akhir dibaca oleh CLI.

c. Modifikasi Antarmuka (CLI): Pada fungsi utama bagian interface, hasil pencarian jalur akan langsung disaring

menggunakan logika percabangan *if-else* sederhana untuk mencetak pesan penolakan yang ramah sebelum ditampilkan ke layar pengguna.

```python
# Memanggil fungsi dijkstra dan rekonstruksi jalur
jarak_peta, parent_peta = dijkstra(kampus_graph, start_node)
rute_terpilih = reconstruct_path(parent_peta, start_node, target_node)

# Penanganan user-friendly tanpa mengubah kompleksitas Big-O dasar
if not rute_terpilih:
    print("\n" + "="*50)
    print(f"⚠️  NAVIGASI GAGAL: Gedung tujuan ({target_node}) terisolasi.")
    print("Sistem tidak dapat menemukan jalur yang terhubung ke lokasi tersebut.")
    print("="*50)
else:
    print(f"✅ Rute berhasil ditemukan: {' -> '.join(rute_terpilih)}")
    print(f"Total Jarak: {jarak_peta[target_node]} meter")
```

d. Analisis Kompleksitas: Modifikasi ini tidak memengaruhi kompleksitas waktu atau ruang graf dasar karena pengecekan kondisi pada dictionary hanya membutuhkan waktu konstan O(1). Performa total sistem navigasi pada kondisi terburuk (worst-case) tetap aman pada skala $O(V^2+E)$.

5. BST menggunakan kode gedung sebagai key pencarian. Jika data dimasukkan secara berurutan seperti A1, A2, A3, dan seterusnya, maka BST akan menjadi linear seperti linked list karena setiap node baru selal ditempatkan di sisi kanan tree. Dampaknya, proses pencarian menjadi lebih lambat karena node harus diperiksa satu per satu. Kompleksitas pencarian yang awalnya dapat berubah menjadi Salah satu solusi untuk mengurangi masalah tersebut adalah memasukkan data gedung secara acak atau tidak berurutan agar bentuk BST lebih seimbang dan proses pencarian menjadi lebih efisien. Contoh:

   Jika data dimasukkan berurutan, maka BST menjadi:

```
A1
 \
  A2
   \
    A3
```

   Tree menjadi linear karena semua node masuk ke kanan.

   - Jika data dimasukkan tidak berurutan:

```
   A3
  /   \
 A1    A4
  \
   A2
```

   - Atau secara acak

```
        A5
       /   \
    A2       A8
   / \       / \
A1 A3  A7 A9
```

Struktur ini lebih baik karena pencarian tidak perlu melewati semua node satu per satu dan BST menjadi lebih seimbang

BAB VII
PENUTUP

A. Kesimpulan

Berdasarkan perancangan, analisis, dan pengujian yang telah dilakukan, dapat diambil beberapa kesimpulan sebagai berikut:

1. Efisiensi Navigasi kampus, Sistem navigasi yang dikembangkan berhasil menggantikan peta konvensional dengan memberikan kalkulasi matematis yang akurat untuk pencarian rute terbaik sehingga mampu menjawab tantangan kebutuhan tata ruang kampus bagi dosen, mahasiswa, maupun pengunjung secara efisien.
2. Optimalisasi Struktur Data,
   - Representasi peta menggunakan Graph dengan Adjacency List (30 gedung dan 44 jalan) terbukti sangat efisien dalam menghemat memori untuk karakteristik peta kampus yang cenderung jarang.
   - Implementasi Queue dan Stack berbasis Linked List berhasil mengoptimalkan manipulasi pointer dengan kompleksitas waktu konstan O(1) tanpa resiko realokasi memori.
   - Struktu data Binary Serach Tree memberikan efisiensi tinggi dalam pencarian direktori gedung berdasarkan kode dengan kompleksitas rata-rata O(log n)
3. Efektivitas Algoritma Pencarian Jalur dan Penelusuran,
   - Algoritma Dijkstra dengan kompleksitas $O(V^2 + E)$ berhasil diimplementasikan secara murni untuk menemukan jalur terpendek dan tercepat antar-gedung secara akurat.
   - Algoritma DFS (Depth First Search) efektif digunakan untuk menyimulasikan rute patroli berkeliling kampus karena karakteristik penelusurannya yang mendalam hingga cabang terakhir.
   - Algoritma BFS (Breadth First Search) berhasil dioptimalkan untuk fitur pengecekan gedung terisolasi. Melalui pengujian kedekatan radial dari Gerbang Utama (A1), sistem membuktikan bahwa seluruh area kampus saling terhubung (Connected Graph) tanpa adanya gedung yang terisolasi.

Secara keseluruhan, integrasi struktur data dan algoritma ini menghasilkan sistem navigasi Smart Campus yang tangguh, memiliki performa waktu (Big-O) yang aman pada kondisi terburuk (worst-case), serta siap dijadikan referensi dalam perkembangan teknologi tata ruang digital ke depan